

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate

7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
-------------------------	----------------------------	--------------------------	---	-----------------------

I AM SANDRA SUDEVAN -192424422 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Sabdra sudevan

Signature of the student:

RUBRICS FOR CALCULATION:

	correct final answer			
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

```

mysql> USE college;
Database changed
mysql> CREATE TABLE DEPARTMENT (
  ->     Dept_ID VARCHAR(10) PRIMARY KEY,
  ->     Dept_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE STUDENT (
  ->     Student_ID VARCHAR(10) PRIMARY KEY,
  ->     Student_Name VARCHAR(50),
  ->     Dept_ID VARCHAR(10),
  ->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE COURSE (
  ->     Course_ID VARCHAR(10) PRIMARY KEY,
  ->     Course_Name VARCHAR(50),
  ->     Dept_ID VARCHAR(10),
  ->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
  -> );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO DEPARTMENT VALUES
  -> ('D01','CSE'),
  -> ('D02','ECE'),
  -> ('D03','EEE');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
  -> ('S101','Arun','D01'),
  -> ('S102','Priya','D02'),
  -> ('S103','Kiran','D01'),
  -> ('S104','Meena','D03');
Query OK, 4 rows affected (0.01 sec)

```

```

mysql>
mysql> INSERT INTO STUDENT VALUES
  -> ('S101','Arun','D01'),
  -> ('S102','Priya','D02'),
  -> ('S103','Kiran','D01'),
  -> ('S104','Meena','D03');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO COURSE VALUES
  -> ('C101','DBMS','D01'),
  -> ('C102','Operating Systems','D01'),
  -> ('C201','Digital Electronics','D02'),
  -> ('C301','Power Systems','D03');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM DEPARTMENT;
+-----+
| Dept_ID | Dept_Name |
+-----+
| D01     | CSE       |
| D02     | ECE       |
| D03     | EEE       |
+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| Student_ID | Student_Name | Dept_ID |
+-----+-----+-----+
| S101       | Arun         | D01     |
| S102       | Priya        | D02     |
| S103       | Kiran        | D01     |
| S104       | Meena        | D03     |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM COURSE;

```

```
mysql> SELECT Student_Name
-> FROM STUDENT
-> WHERE Dept_ID = (
->     SELECT Dept_ID
->     FROM DEPARTMENT
->     WHERE Dept_Name = 'CSE'
-> );
```

Student_Name
Arun
Kiran

2 rows in set (0.03 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
-> FROM STUDENT s
-> RIGHT JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kiran	CSE
Priya	ECE
Meena	EEE

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM COURSE;
```

Course_ID	Course_Name	Dept_ID
C101	DBMS	D01
C102	Operating Systems	D01
C201	Digital Electronics	D02
C301	Power Systems	D03

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM COURSE;
```

Course_ID	Course_Name	Dept_ID
C101	DBMS	D01
C102	Operating Systems	D01
C201	Digital Electronics	D02
C301	Power Systems	D03

4 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
-> FROM STUDENT s
-> INNER JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kiran	CSE
Priya	ECE
Meena	EEE

4 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
-> FROM STUDENT s
-> LEFT JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Priya	ECE
Kiran	CSE
Meena	EEE

4 rows in set (0.03 sec)

```
mysql> SELECT Student_Name
```

```
mysql> SELECT Course_Name
-> FROM COURSE
-> WHERE Dept_ID = (
->     SELECT Dept_ID
->     FROM DEPARTMENT
->     WHERE Dept_Name = 'CSE'
-> );
```

```
+-----+
| Course_Name |
+-----+
| DBMS        |
| Operating Systems |
+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT Dept_Name
-> FROM DEPARTMENT
-> WHERE Dept_ID IN (
->     SELECT Dept_ID
->     FROM COURSE
-> );
```

```
+-----+
| Dept_Name |
+-----+
| CSE       |
| ECE       |
| EEE       |
+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT s.Student_Name,
->     d.Dept_Name,
->     c.Course_Name
-> FROM STUDENT s
-> JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID
-> JOIN COURSE c
-> ON d.Dept_ID = c.Dept_ID;
```

```
+-----+-----+-----+
| Student_Name | Dept_Name | Course_Name |
+-----+-----+-----+
| Arun         | CSE       | DBMS        |
| Arun         | CSE       | Operating Systems |
| Kiran        | CSE       | DBMS        |
| Kiran        | CSE       | Operating Systems |
| Priya        | ECE       | Digital Electronics |
| Meena        | EEE       | Power Systems |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT s.Student_Name,
->         d.Dept_Name,
->         c.Course_Name
-> FROM STUDENT s
-> JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID
-> JOIN COURSE c
-> ON d.Dept_ID = c.Dept_ID;
```

Student_Name	Dept_Name	Course_Name
Arun	CSE	DBMS
Arun	CSE	Operating Systems
Kiran	CSE	DBMS
Kiran	CSE	Operating Systems
Priya	ECE	Digital Electronics
Meena	EEE	Power Systems

```
6 rows in set (0.00 sec)

mysql>
```

2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse

$$\pi_{\text{Student_Name, Course_Name}} \left(\sigma_{\text{Dept_ID}='D01' \text{ AND } \text{Course_Name}='DBMS'} \left(\text{STUDENT} \bowtie \text{COURSE} \right) \right)$$

3: Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse

```
mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| Course_ID | Course_Name      | Dept_ID |
+-----+-----+-----+
| C101      | DBMS              | D01     |
| C102      | Operating Systems | D01     |
| C201      | Digital Electronics | D02     |
| C301      | Power Systems     | D03     |
+-----+-----+-----+
4 rows in set (0.004 sec)
```

```
mysql>
mysql> SELECT s.Student_Name,d.Dept_Name
-> FROM STUDENT s
-> INNER JOIN DEPARTMENT d
-> ON s.Dept_ID=d.Dept_ID;
+-----+-----+
| Student_Name | Dept_Name |
+-----+-----+
| Arun         | CSE       |
| Kavın        | CSE       |
| Divya        | ECE       |
| Meena        | EEE       |
+-----+-----+
4 rows in set (0.013 sec)
```

```
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Gender | DOB          | MobileNo | City |
| Dept_ID   |
+-----+-----+-----+-----+-----+-----+
+-----+
| S101      | Arun         | M      | 2004-05-12  | 9876543210 | Chennai |
| D01       |
| S102      | Divya        | F      | 2005-02-18  | 9876543211 | Madurai |
| D02       |
| S103      | Kavın        | M      | 2004-11-30  | 9876543212 | Coimbatore |
| D01       |
| S104      | Meena        | F      | 2005-08-25  | 9876543213 | Salem  |
| D03       |
+-----+-----+-----+-----+-----+-----+
+-----+
4 rows in set (0.005 sec)
```

```
mysql>
mysql> SELECT s.Student_Name,d.Dept_Name
      -> FROM STUDENT s
      -> LEFT JOIN DEPARTMENT d
      -> ON s.Dept_ID=d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Divya	ECE
Kavin	CSE
Meena	EEE

```
4 rows in set (0.004 sec)
```

```
mysql>
mysql> SELECT s.Student_Name,d.Dept_Name
      -> FROM STUDENT s
      -> INNER JOIN DEPARTMENT d
      -> ON s.Dept_ID=d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kavin	CSE
Divya	ECE
Meena	EEE

```
4 rows in set (0.013 sec)
```

```
mysql>
mysql> SELECT s.Student_Name,d.Dept_Name
      -> FROM STUDENT s
      -> RIGHT JOIN DEPARTMENT d
      -> ON s.Dept_ID=d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kavin	CSE
Divya	ECE
Meena	EEE

```
4 rows in set (0.005 sec)
```

```
mysql>
mysql> SELECT Course_Name
-> FROM COURSE
-> WHERE Dept_ID=
-> (
-> SELECT Dept_ID
-> FROM DEPARTMENT
-> WHERE Dept_Name='CSE'
-> );
```

```
+-----+
| Course_Name |
+-----+
| DBMS        |
| Operating Systems |
+-----+
2 rows in set (0.004 sec)
```

```
mysql>
mysql> SELECT Student_Name
-> FROM STUDENT
-> WHERE Dept_ID=
-> (
-> SELECT Dept_ID
-> FROM DEPARTMENT
-> WHERE Dept_Name='CSE'
-> );
```

```
+-----+
| Student_Name |
+-----+
| Arun         |
| Kevin        |
+-----+
2 rows in set (0.017 sec)
```

```
mysql>
mysql> SELECT Dept_Name
-> FROM DEPARTMENT
-> WHERE Dept_ID IN
-> (
-> SELECT Dept_ID
-> FROM COURSE
-> );
```

Dept_Name
CSE
ECE
EEE

```
3 rows in set (0.010 sec)
```

```
mysql>
mysql> SELECT s.Student_Name,
-> d.Dept_Name,
-> c.Course_Name
-> FROM STUDENT s
-> JOIN DEPARTMENT d
-> ON s.Dept_ID=d.Dept_ID
-> JOIN COURSE c
-> ON d.Dept_ID=c.Dept_ID;
```

Student_Name	Dept_Name	Course_Name
Kavin	CSE	DBMS
Arun	CSE	DBMS
Kavin	CSE	Operating Systems
Arun	CSE	Operating Systems
Divya	ECE	Digital Electronics
Meena	EEE	Power Systems

```
6 rows in set (0.014 sec)
```

```
mysql>
mysql> UPDATE STUDENT
    -> SET Student_Name='Arun Kumar'
    -> WHERE Student_ID='S101';
Query OK, 1 row affected (0.020 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT * FROM STUDENT;
```

Student_ID	Student_Name	Gender	DOB	MobileNo	City
S101	Arun Kumar	M	2004-05-12	9876543210	Chennai
S102	Divya	F	2005-02-18	9876543211	Madurai
S103	Kavin	M	2004-11-30	9876543212	Coimbatore
S104	Meena	F	2005-08-25	9876543213	Salem

```
4 rows in set (0.004 sec)
```

```
mysql>
mysql> CREATE VIEW Student_View AS
    -> SELECT Student_ID,Student_Name,City
    -> FROM STUDENT;
Query OK, 0 rows affected (0.058 sec)

mysql>
mysql> SELECT * FROM Student_View;
```

Student_ID	Student_Name	City
S101	Arun Kumar	Chennai
S102	Divya	Madurai
S103	Kavin	Coimbatore
S104	Meena	Salem

```
4 rows in set (0.023 sec)
```

```
mysql>
mysql> CREATE INDEX idx_dept
      -> ON STUDENT(Dept_ID);
Query OK, 0 rows affected (0.754 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT Dept_ID,
      -> COUNT(Student_ID) AS Total_Students
      -> FROM STUDENT
      -> GROUP BY Dept_ID;
```

Dept_ID	Total_Students
D01	2
D02	1
D03	1

```
3 rows in set (0.105 sec)
```

4: Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze

```
mysql>
mysql> SELECT Student_Name
      -> FROM STUDENT
      -> WHERE Dept_ID =
      -> (
      ->     SELECT Dept_ID
      ->     FROM DEPARTMENT
      ->     WHERE Dept_Name='CSE'
      -> );
```

Student_Name
Arun
Kavin

```
2 rows in set (0.004 sec)
```

```
mysql>
mysql> SELECT Dept_ID
      -> FROM DEPARTMENT
      -> WHERE Dept_Name='CSE';
```

Dept_ID
D01

```
1 row in set (0.002 sec)
```

5: Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze

```
mysql> SELECT s.Student_Name,d.Dept_Name
      -> FROM STUDENT s
      -> INNER JOIN DEPARTMENT d
      -> ON s.Dept_ID=d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kavin	CSE
Divya	ECE
Meena	EEE

```
4 rows in set (0.008 sec)
```

```
mysql>
mysql> SELECT Student_Name
      -> FROM STUDENT
      -> WHERE Dept_ID=
      -> (
      -> SELECT Dept_ID
      -> FROM DEPARTMENT
      -> WHERE Dept_Name='CSE'
      -> );
```

Student_Name
Arun
Kavin

```
2 rows in set (0.012 sec)
```

```
mysql>
mysql> SELECT * FROM STUDENT;
```

Student_ID	Student_Name	Dept_ID
S101	Arun	D01
S102	Divya	D02
S103	Kavin	D01
S104	Meena	D03

```
4 rows in set (0.005 sec)
```

```
mysql> SELECT * FROM DEPARTMENT;
```

Dept_ID	Dept_Name
D01	CSE
D02	ECE
D03	EEE

```
3 rows in set (0.008 sec)
```

6.Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create

```
mysql>
mysql> SELECT * FROM STUDENT_VIEW;
```

Student_ID	Student_Name	City
S101	Arun	Chennai
S102	Divya	Madurai
S103	Kavin	Coimbatore
S104	Meena	Salem

```
4 rows in set (0.013 sec)
```

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze

Relational Algebra

π Student_Name
(
 σ Dept_ID='D01'
(STUDENT)
)

Equivalent SQL

USE college;

SELECT Student_Name
FROM STUDENT
WHERE Dept_ID='D01';

SELECT *
FROM STUDENT
WHERE Dept_ID='D01';

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate

```
mysql>
mysql> SELECT Student_Name
      -> FROM STUDENT
      -> WHERE Dept_ID='D01';
+-----+
| Student_Name |
+-----+
| Arun         |
| Kevin        |
+-----+
2 rows in set (0.011 sec)
```

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create

```
mysql>
mysql> SELECT Dept_ID,
        -> COUNT(Student_ID) AS Total_Students
        -> FROM STUDENT
        -> GROUP BY Dept_ID;
```

Dept_ID	Total_Students
D01	2
D02	1
D03	1

```
3 rows in set (0.030 sec)
```

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate

```
mysql> SELECT s.Student_Name, d.Dept_Name
        -> FROM STUDENT s
        -> JOIN DEPARTMENT d
        -> ON s.Dept_ID=d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kavin	CSE
Divya	ECE
Meena	EEE

```
4 rows in set (0.008 sec)
```

```
mysql> SELECT Student_Name
-> FROM STUDENT
-> WHERE Dept_ID=
-> (
-> SELECT Dept_ID
-> FROM DEPARTMENT
-> WHERE Dept_Name='CSE'
-> );
```

```
+-----+
| Student_Name |
+-----+
| Arun         |
| Kavın        |
+-----+
```

```
2 rows in set (0.009 sec)
```

```
mysql>
mysql> SELECT * FROM STUDENT_VIEW;
```

```
+-----+-----+-----+
| Student_ID | Student_Name | Dept_ID |
+-----+-----+-----+
| S101       | Arun         | D01     |
| S102       | Divya        | D02     |
| S103       | Kavın        | D01     |
| S104       | Meena        | D03     |
+-----+-----+-----+
```

```
4 rows in set (0.015 sec)
```